Lecture 7: Model Selection & Assessment

New terminology

- "Validation set" – a data set, like the training and test sets, but that's used to evaluate the model/algorithm used
  - Can be a subset of the training set (as we saw last time)
  - Could also be a former test set:
    - You make a self-driving car using a bunch of training data
    - You set the car up on a course, and how it behaves on this course is the test data
    - Now, take that test data to evaluate your car, it's the validation set
    - If it passes validation, toss the validation set and put it on the road with the user experience being the new test sets

Cross validation

- As we talked about last time, cross validation (or cross-validation) is a way to get example test data by using a fraction of training data as test data (while removing that fraction from your training data)

- It works by separating the data into “groups” and evaluating by training on all groups except one and then testing on that one. You would then switch the “one” that is being used as the test until all groups have been test groups

How many groups should you have? Or, put another way, how many “pieces” of data should be in each group?

Well, let’s take the extremes

- Have the training set be a single piece of data, validation set will be all the other pieces. Then, repeat by switching what the one piece is and averaging.
  - Is this good? Why or why not?

- Have the training set by all data except a single piece, validation set will be a single piece of data. Then, repeat by switching what the one piece is and averaging.
  - Is this good? Why or why not?

Leave-one-out (LOO) cross validation

- Consider a validation set with 1 example:
  - D – training data
  - D_j – training data with the jth data point (x_j, y_j) moved to the validation set
- Learn classifier f_{Dj} with D_j dataset
- Estimate the error based on predicting y_j
  - This is an unbiased estimate of the true error of f_{Dj}, which should be ~f_D

Leave-one-out (LOO) cross validation

- Okay, I've got an estimate based on a nearly complete training dataset, but the estimate is based on a validation set of size 1, soo…
- LOO cross validation – average over all possible data points, j:
  - For each j left out, build a new classifier/algorithm f_{Dj}
  - Estimate the error as:

$$\text{error}_{LOO} = \frac{1}{n}\sum_{j=1}^{n}(y_j - f_{\mathcal{D}\setminus j}(x_j))^2$$

Or, the average squared difference between the measurement and the prediction

Leave-one-out (LOO) cross validation

- LOO cross validation (LOOCV) is an *almost* unbiased estimate of the true error (very, *very* close in most cases)
  - You are measuring your error in an unbiased way for f_{Dj}, but it isn't exactly equal to your error for f_D
  - So, it isn't an estimate of the true error if you had all N data points (but if N is large, it is very, *very* close)

- LOO is an extremely powerful, virtually unbiased method for model selection. Feel free to use it for almost any application!

Too good to be true…

- What’s the computational cost of LOO/LOOCV?
- Suppose you:
  - Have 100,000 data points
  - Have an efficient machine learning algorithm that only takes 1 second
  - How long would the validation take?

LOOCV

- A lot of this class will be:
  - “Wow, this is great!”
  - “Well, it doesn’t actually work that well with real data, is slow, etc.”
  - Cross validation is powerful but slow, and as such has many, many potential applications, particularly:
    - Not that much data
    - You don’t care about the time to test your algorithm (say, you want to publish your algorithm)

k-fold Cross Validation

- *Randomly* divide data into *k* equal parts
  - D_1, D_2, …, D_k
- For each i from 1 to k:
  - Learn classifier $f_{D\backslash Di}$ using all data except data in D_i
  - Estimate error of $f_{D\backslash Di}$ on the validation set, D_i:

$$\text{error}_{\mathcal{D}_i} = \frac{1}{|\mathcal{D}_i|} \sum_{(x_j, y_j) \in \mathcal{D}_i} (y_j - f_{\mathcal{D}\backslash\mathcal{D}_i}(x_j))^2$$

$$error_{k-fold} = \frac{1}{k} \sum_{i=1}^{k} error_{\mathcal{D}_i}$$

k-fold Cross Validation

- *k*-fold cross validation is:
  - Much faster to compute than LOO
  - More biased (uses much less data)
  - Depends on user to select *k* (10 is pretty standard but can vary quite a bit)

Limitations of cross validation

- You have relatively little data, say 20 examples
  - *k*-fold cross validation is a bad choice since there won't be much validation data in any one error test
  - LOO would work, but what if you want to know the error for a *particular example x* instead of the whole data?

Bootstrapping

- Given a set of data that are independent and identically distributed (i.i.d.) samples, D = {Z_1, Z_2,…Z_n}, we want to compute θ

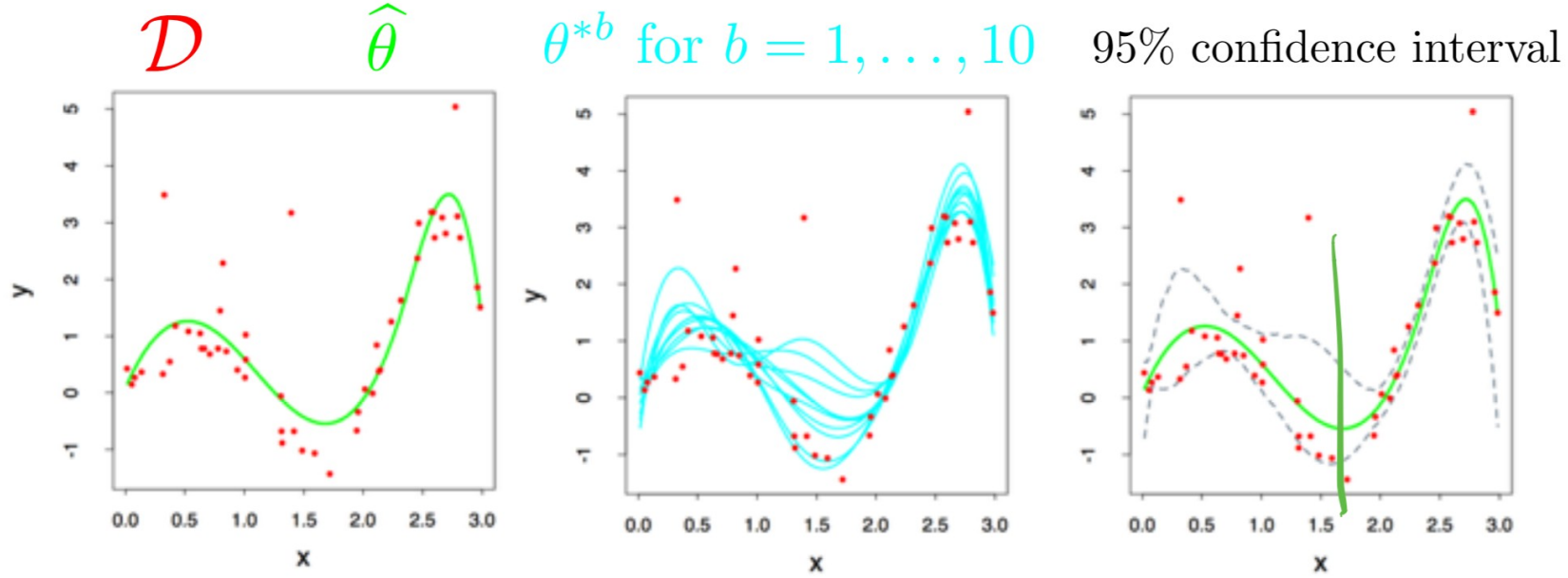
- For b=1, 2,…B, we define the *b*th bootstrapped dataset as drawing n samples ***with replacement*** from D

$$\mathcal{D} = \{z_1, \ldots, z_n\} \overset{i.i.d.}{\sim} F_Z \qquad \widehat{\theta} = t(\mathcal{D})$$

$$\mathcal{D}^{*b} = \{z_1^{*b}, \ldots, z_n^{*b}\} \overset{i.i.d.}{\sim} \widehat{F}_{Z,n} \quad \theta^{*b} = t(\mathcal{D}^{*b})$$

Bootstrap applications

- Common applications of the bootstrap:
  - Estimate parameters that escape simple analysis like the variance or median of an estimate
  - Confidence intervals
  - Estimates of error for a particular example:



Bootstrap takeaways

- Advantages:
  - Bootstrap is very generally applicable. Build a confidence interval around anything
  - Very simple to use
  - Appears to give meaningful results even when the amount of data is very small
  - Very strong asymptotic theory (as num. examples goes to infinity)

Bootstrap takeaways

- Disadvantages:
  - Very few meaningful finite-sample guarantees
  - Potentially computationally intensive
  - Reliability relies on test statistic and rate of convergence of empirical CDF to true CDF, which is unknown
  - Poor performance on “extreme statistics” (e.g., the max)